VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Pham Trung Kien

GRAMMAR-BASED GREYBOX FUZZING FOR DBMS VULNERABILITY DETECTION

BACHELOR'S THESIS

Major: Information Technology

Supervisor: PhD. Le Dinh Thanh

HANOI - 2026

Statement of Integrity

I hereby declare that the graduation thesis entitled "Grammar-based Greybox Fuzzing for DBMS Vulnerability Detection" presented in this report is entirely my own work. The content does not involve any form of plagiarism or the use of others' results without proper citation. The proposed method in this thesis is the result of my own research, conducted under the supervision of PhD. Le Dinh Thanh. I take full responsibility for any violations of the regulations of the University of Engineering and Technology, Vietnam National University, Hanoi.

Ha Noi, June 2026

Student

Pham Trung Kien

Acknowledgements

I would like to express my sincere gratitude to the Faculty of Information Technology, University of Engineering and Technology, Vietnam National University, Hanoi. The knowledge, skills, and perspectives I have gained throughout four years of study here form the foundation upon which this thesis is built. The dedication of the faculty and staff in providing a rigorous and supportive academic environment has been invaluable to my development as an engineer.

I am deeply grateful to my supervisor, PhD. Le Dinh Thanh, for his patient guidance, insightful feedback, and continuous encouragement throughout the entire research process. His expertise in systems security and his willingness to engage with the technical challenges of this project—from grammar design to experimental evaluation—shaped the direction and quality of this work in ways I could not have achieved independently.

Finally, I would like to thank my classmates in the K67CS cohort for their companionship, intellectual exchange, and mutual support over the course of our studies. The discussions, shared struggles, and moments of collaboration we experienced together made this journey not only more productive but genuinely enjoyable.

Abstract

Database management systems (DBMS) are critical infrastructure components, and their vulnerabilities represent serious security threats. Automated vulnerability detection through fuzzing is a promising approach, but existing grammar-based fuzzers apply uniform random sampling over production rules, wasting mutation budget on low-value syntactic paths and failing to prioritize the structural patterns most likely to expose bugs.

This thesis presents a grammar-based greybox fuzzing system targeting SQLite. A structural primitives grammar of 475 rules encodes SQL patterns known to trigger vulnerability classes—including integer overflow in expression trees, use-after-free in query planner optimizations, and memory corruption in aggregate processing—without hardcoding specific proof-of-concept inputs. On top of this grammar, a multi-armed bandit controller using the EXP3 algorithm adaptively selects grammar rules based on coverage feedback from an AFL-compatible fork server, aiming to concentrate mutations on rules that generate coverage-increasing inputs.

Experiments across 53 campaigns on 3 SQLite versions containing 6 known CVEs show that uniform sampling discovers significantly more unique crash classes than the bandit controller (mean 8.6 vs. 4.2 per campaign, $p \approx 0.01$, Mann-Whitney U test). Analysis reveals that the bandit converges to exploitation of a small rule subset, starving exploration and reducing grammatical diversity. A grammar gap analysis further shows that 4 of the 6 target CVEs were unreachable until structural primitives for window functions and self-referential generated columns were added in grammar version 3.2, demonstrating that grammar coverage is the dominant factor for vulnerability reachability.

Keywords: Grammar-based Fuzzing, Greybox Fuzzing, DBMS Security, Vulnerability Detection, Multi-Armed Bandit

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Form	Description
AFL	American Fuzzy Lop	Coverage-guided greybox fuzzer that uses shared-memory bitmaps to track branch coverage
ASan	AddressSanitizer	Compiler instrumentation tool for detecting memory errors such as buffer overflows and use-after-free
CFG	Context-Free Grammar	Formal grammar used to specify the syntactic structure of inputs for grammar-based fuzzing
CVE	Common Vulnerabilities and Exposures	Standardized identifier assigned to publicly disclosed security vulnerabilities
DBMS	Database Management System	Software system responsible for storing, retrieving, and managing structured data
DQN	Deep Q-Network	Deep reinforcement learning algorithm that approximates the Q-value function using a neural network
FTS	Full-Text Search	SQLite extension providing full-text indexing and search capabilities over text columns
MAB	Multi-Armed Bandit	Online learning framework for sequential decision-making under uncertainty with exploration-exploitation trade-off
RL	Reinforcement Learning	Machine learning paradigm in which an agent learns a policy by interacting with an environment and receiving reward signals
SQL	Structured Query Language	Declarative language for defining, manipulating, and querying data in

Chapter 1

Introduction

1.1 Context and Motivation

Database management systems occupy a uniquely sensitive position in the modern software stack: they are responsible for storing, indexing, and serving the data on which virtually every application depends, from banking infrastructure to personal health records to critical national systems. A vulnerability in a database engine therefore does not merely threaten one application but potentially every application built on top of it. SQLite in particular stands out as the world's most widely deployed database engine, embedded in every major smartphone operating system, every mainstream web browser, the Python standard library, and countless embedded devices [1]. Its ubiquity makes it an especially consequential attack surface. Despite this importance, and despite extensive manual testing and code review, vulnerabilities continue to be discovered in SQLite at a steady pace. Between 2019 and 2020 alone, six Common Vulnerabilities and Exposures (CVEs) were filed against SQLite, affecting released versions 3.30.1 through 3.32.2: an integer overflow in the `INTERSECT` operator (CVE-2019-19646 [2]), a stack overflow in expression parsing (CVE-2020-13434 [3]), a null pointer dereference in the `isAggregateFunction` routine (CVE-2020-9327 [4]), a null pointer dereference in the recursive `isAggregateFunction` (CVE-2020-13435 [5]), a use-after-free triggered by the `ALTER TABLE` rename mechanism (CVE-2020-13871 [6]), and a heap buffer overflow in multi-threaded `SELECT` with window functions (CVE-2020-15358 [7]). The persistence of such defects in a heavily scrutinised, mature codebase underscores the need for more systematic automated approaches to vulnerability discovery.

Fuzzing has emerged as one of the most productive techniques for automated vulnerability detection, responsible for discovering thousands of bugs across the software industry [8]. The central idea is to generate large volumes of semi-random inputs, feed them to a program under test, and observe whether the program crashes, hangs, or violates a safety property enforced by a dynamic analysis tool such as AddressSanitizer [9]. Coverage-guided greybox fuzzers like AFL [10] and libFuzzer [11] refine this by instrumenting the target binary to track which code branches are exercised by each input, then retaining

inputs that increase coverage for further mutation. This feedback loop allows the fuzzer to progressively steer execution toward unexplored regions of the program. In practice, coverage-guided greybox fuzzing has proven highly effective for file parsers, network protocol implementations, and similar targets where input structure is loose or where arbitrary byte sequences can still reach meaningful program states. However, such byte-level mutation fuzzers encounter a fundamental obstacle when applied to programs that expect structured inputs, such as database engines that parse SQL. Random byte flips, bit-flips, and splice operations overwhelmingly produce inputs that are rejected at the parser stage, long before they can reach the deeper, semantically complex logic where vulnerabilities typically reside. The result is that the fuzzer saturates shallow code coverage quickly and fails to exercise the query optimiser, the window function engine, the virtual table subsystem, or other high-complexity components.

Grammar-based fuzzing addresses this problem by generating inputs from a formal specification of the target language, most commonly a context-free grammar, thereby guaranteeing syntactic validity and enabling the fuzzer to reach deeper program logic from the very first input. Nautilus [12] represents a particularly influential instance of this paradigm: it combines grammar-based generation with AFL-style coverage feedback, maintaining a corpus of grammar derivation trees rather than raw byte strings, and mutating those trees through subtree splicing, rule replacement, and random regeneration. This approach achieves substantially deeper code coverage on structured-input targets than byte-level fuzzers. Other grammar-aware fuzzers occupy adjacent points in the design space. Superion [13] extends AFL with grammar-aware mutations for JavaScript and XML targets. Grimoire [14] synthesises approximate structural patterns without requiring an explicit grammar specification, instead generalising from inputs that achieve new coverage. For relational databases specifically, Squirrel [15] generates SQL queries with explicit awareness of language validity constraints and coverage feedback, while SQLsmith [16] takes a simpler approach of generating random but syntactically valid SQL queries without coverage guidance.

A limitation shared by all of these grammar-based approaches, however, is the strategy used to select which production rule to apply at each step of input generation. Existing systems uniformly sample from all applicable rules with equal probability, meaning that a rule producing a common `SELECT` statement with a simple `WHERE` clause competes on equal footing with a rule producing a window function frame specification or a subquery

inside a `CHECK` constraint. In a grammar with hundreds of rules, uniform sampling is necessarily diffuse: a large fraction of the mutation budget is consumed by rules that generate inputs which exercise already-covered code paths, yielding no new coverage signal and contributing nothing toward the discovery of new vulnerability classes. The uniform policy is stateless and memoryless, incapable of learning from the history of which rules have been productive. This inefficiency motivates the investigation of adaptive rule selection strategies that direct the mutation budget toward grammar rules with demonstrated potential to expand coverage.

1.2 Problem Statement

This thesis investigates whether adaptive rule selection can improve the effectiveness of grammar-based greybox fuzzing for vulnerability discovery in database management systems. The central research question is: can a multi-armed bandit algorithm [17] that learns from coverage feedback to prioritise grammar production rules discover more diverse crash classes than uniform random sampling, when applied to a grammar-based fuzzer targeting SQLite?

The bandit hypothesis is that adaptive sampling, by concentrating mutation effort on rules that have historically expanded coverage, should guide the fuzzer toward unexplored program regions more efficiently than uniform sampling, thereby increasing both the speed and diversity of vulnerability discovery. The counter-hypothesis, which the experimental results of this thesis will ultimately support, is that such exploitation of historically productive rules may simultaneously reduce the structural diversity of the generated input corpus. Vulnerability discovery in mature software often requires rare, compositionally unusual inputs that exercise corner cases involving uncommon SQL constructs — precisely the inputs that an exploitation-focused bandit policy may systematically underweight. The interplay between coverage exploitation and structural diversity therefore constitutes the core tension this thesis examines.

1.3 Objectives and Scope

This thesis pursues three interconnected objectives, each building on the previous. The first objective is to design and implement a grammar-based greybox fuzzing system for SQLite that encodes SQL patterns known to trigger specific vulnerability classes without

reproducing proof-of-concept exploit inputs verbatim. This requires a structural primitives methodology in which the grammar decomposes SQL into layered non-terminal categories — schema setup operations, stress queries involving complex joins and aggregations, validation operations that exercise integrity checking, and boundary function calls that probe edge cases in built-in SQL functions — such that the compositional combination of these primitives can independently rediscover vulnerability-triggering inputs. The grammar evolved through three versions (v3.0 to v3.2), growing from 449 to 475 production rules, and targets the six CVEs identified above across four SQLite releases.

The second objective is to integrate a multi-armed bandit algorithm, specifically the EXP3 algorithm [17], into the grammar rule selection mechanism of the Nautilus fuzzer [12]. EXP3 (Exponential-weight algorithm for Exploration and Exploitation) is designed for adversarial bandit settings where reward distributions may be non-stationary — a natural fit for fuzzing, where the coverage landscape changes as the corpus evolves. The integration requires implementing O(1) weighted sampling via the alias method [18], a coverage reward signal derived from new bitmap edges discovered per generated input, and an exponential moving average normalisation scheme to prevent weight compounding over long campaigns.

The third objective is to conduct controlled empirical experiments comparing the bandit-guided and uniform sampling policies across multiple SQLite versions with confirmed CVEs, using a rigorous statistical methodology. The experimental scope encompasses 53 fuzzing campaigns of 30 minutes each across the four target SQLite versions (3.30.1, 3.31.1, 3.32.0, and 3.32.2) and two grammar variants, with crash deduplication by stack hash and statistical comparison using the Mann-Whitney U test [19] to assess whether observed differences in crash class counts are statistically significant.

1.4 Contributions

This thesis makes three primary contributions to the area of grammar-based fuzzing for database vulnerability detection. The first contribution is a structural primitives grammar design methodology for SQL-based fuzzing. Rather than encoding specific SQL statements known to trigger bugs, the methodology decomposes SQL into layered non-terminals — `Schema-Setup`, `Stress-Query`, `Validation-Op`, and `Boundary-Func` — that capture the structural patterns associated with vulnerability classes without contaminating the fuzzer's search with proof-of-concept bias. This compositional design allows the fuzzer

to independently reconstruct vulnerability-triggering inputs from primitive combinations, generalise across SQLite versions, and avoid the common failure mode where a grammar is so specific to known bugs that it fails to discover new ones. The grammar's three-version evolution (v3.0 to v3.2, 449 to 475 rules) demonstrates how iterative refinement guided by coverage and crash evidence can progressively improve a structural grammar.

The second contribution is the integration of EXP3 multi-armed bandit adaptive sampling into grammar-based greybox fuzzing, with O(1) weighted sampling via the alias method [18]. The implementation includes a coverage-derived reward signal, exponential moving average normalisation, and a corrected delta computation that properly credits the bandit for newly discovered bitmap edges rather than compounding weight updates. This constitutes the first integration of EXP3 adaptive sampling into a Nautilus-class grammar fuzzer with a formal evaluation against a controlled baseline.

The third contribution is an empirical finding that challenges the intuitive appeal of adaptive sampling for vulnerability discovery. Across 53 campaigns, uniform random sampling discovers approximately twice as many unique crash classes as bandit-guided sampling: a mean of 8.6 crash classes versus 4.2 for the bandit ($p \approx 0.01$, Mann-Whitney U test [19]). This result demonstrates that structural diversity — the breadth of SQL constructs exercised — is more important than coverage exploitation for discovering CVE-class vulnerabilities in SQLite, and provides empirical grounding for the claim that exploration outperforms exploitation in this setting. The result also reveals a previously undocumented failure mode of bandit-guided fuzzing: weight concentration on a small subset of highly rewarded rules collapses input diversity and causes the fuzzer to miss entire vulnerability classes.

1.5 Thesis Organisation

The remainder of this thesis is organised as follows. Chapter 2 surveys the background literature in four areas: the foundations of coverage-guided greybox fuzzing and grammar-based input generation, the multi-armed bandit formulation and the EXP3 algorithm, the dynamic analysis tools (AddressSanitizer and UndefinedBehaviorSanitizer) used as vulnerability oracles, and the related work on grammar-aware fuzzers and database testing tools most directly relevant to this thesis. Chapter 3 presents the proposed system: the overall architecture of the grammar-based fuzzer targeting SQLite, the structural primitives grammar design methodology, the integration of EXP3 bandit sampling into the rule

selection mechanism, the SQLite harness design and sanitiser oracle configuration, and the crash triage and deduplication pipeline. Chapter 4 reports the experimental evaluation, structured around four research questions covering crash discovery rate, crash class diversity, coverage growth trajectory, and the statistical significance of the bandit-versus-uniform comparison; results are presented for all four SQLite target versions and discussed in light of the original hypotheses. The Conclusion synthesises the key findings, identifies the limitations of the current work, and outlines directions for future research including deeper reinforcement learning integration, grammar generalisation across database engines, and longer-duration campaign studies.

Chapter 2

Background

This chapter establishes the theoretical and technical foundations upon which this thesis builds. We begin with the nature of software vulnerabilities and the Common Vulnerabilities and Exposures (CVE) system, then proceed through fuzzing fundamentals, grammar-based input generation, coverage-guided feedback mechanisms, multi-armed bandit algorithms for adaptive selection, and sanitizer-based bug oracles. The chapter concludes with a survey of related work that positions this thesis within the broader landscape of automated vulnerability discovery.

2.1 Software Vulnerabilities and CVEs

A software vulnerability is a flaw in a program's design, implementation, or configuration that can be exploited to violate the system's security policy. The Common Vulnerabilities and Exposures (CVE) system, maintained by the MITRE Corporation and catalogued in the National Vulnerability Database (NVD), assigns each publicly disclosed vulnerability a unique identifier of the form CVE-YYYY-NNNNN, enabling unambiguous cross-referencing across advisories, patches, and research publications.

Several vulnerability classes are particularly relevant to database management systems implemented in C. Buffer overflows, both heap-based and stack-based, occur when a program writes data beyond the bounds of an allocated memory region, potentially overwriting adjacent data structures or control flow metadata. Use-after-free vulnerabilities arise when a program continues to reference a memory region after it has been deallocated, leading to undefined behavior when the freed region is subsequently reallocated for a different purpose. Integer overflows occur when arithmetic operations produce results that exceed the representable range of the target data type, causing silent wraparound that can cascade into incorrect buffer size calculations. Null pointer dereferences result from attempting to access memory through an invalid (null) pointer, typically causing immediate program termination. Undefined behavior encompasses a broad category of operations that the C language standard leaves unspecified, permitting compilers to generate arbitrary code for such cases.

SQLite [1] exemplifies the intersection of widespread deployment and memory safety challenges. As an embedded database engine written in approximately 150,000 lines of C code, SQLite is deployed in virtually every smartphone, web browser, and operating system worldwide, with an estimated three trillion active installations. Despite rigorous testing infrastructure including the OSSFuzz continuous fuzzing platform, SQLite has a documented history of memory safety vulnerabilities. This thesis targets six CVEs across four SQLite versions, summarized in Table 2.1.

Table 2.1: Summary of target CVEs used in this thesis for evaluating grammar-based fuzzing effectiveness.

CVE ID	**Version**	**Type**	**Description**
CVE-2019-19646 [2]	3.30.1	Infinite loop	`PRAGMA integrity_check` with self-referent
CVE-2020-9327 [4]	3.31.1	Uninitialized pointer	Generated column self-reference
CVE-2020-13434 [3]	3.31.1	Integer overflow	`printf` with `INT32_MAX` precision
CVE-2020-13435 [5]	3.32.0	Null pointer deref	`NATURAL JOIN` with `coalesce` and window
CVE-2020-13871 [6]	3.32.2	Use-after-free	`EXCEPT` with window functions in subquery
CVE-2020-15358 [7]	3.32.2	Heap buffer overread	`INTERSECT` in `WHERE` with `VIEW`

These vulnerabilities span the full spectrum of memory safety issues in C programs: from integer arithmetic errors (CVE-2020-13434) through pointer safety violations (CVE-2020-9327, CVE-2020-13435, CVE-2020-13871) to spatial memory safety failures (CVE-2020-15358) and algorithmic denial-of-service (CVE-2019-19646). Each requires specific SQL constructs to trigger, making them ideal targets for evaluating whether grammar-based fuzzing can systematically discover such patterns.

2.2 Fuzzing

Definition 2.1: *Fuzzing is an automated software testing technique that generates a large number of semi-valid inputs to a program under test, monitors the program's execution for anomalous behavior (crashes, hangs, assertion failures, sanitizer violations), and iteratively refines its input generation strategy based on feedback from prior executions.*

The fuzzing landscape can be taxonomized along two orthogonal axes: the degree of program analysis employed and the input generation strategy. Along the analysis axis,

black-box fuzzers generate inputs without any knowledge of the program's internal structure, relying solely on random mutation or grammar-based generation with no execution feedback. White-box fuzzers employ heavyweight program analysis techniques such as symbolic execution and constraint solving to reason about program paths and generate inputs that satisfy specific path constraints. Greybox fuzzers occupy the middle ground, employing lightweight instrumentation to observe program behavior (typically branch coverage) without the computational expense of full symbolic analysis [8].

Along the generation axis, mutation-based fuzzers begin with a seed corpus of valid inputs and apply random modifications such as bit flips, byte insertions, arithmetic operations on integer fields, and block splicing between seeds. Generation-based fuzzers construct inputs from scratch according to a model of the expected input format, typically a grammar or protocol specification, without requiring seed inputs.

The coverage feedback loop constitutes the central mechanism of modern greybox fuzzing. The program under test is instrumented at compile time to record which code edges are exercised during execution. For each generated input, the fuzzer executes the instrumented program, collects coverage information, and determines whether the input exercised previously unseen edges. Inputs that discover new coverage are retained in the corpus and serve as seeds for subsequent mutations, creating a directed exploration of the program's state space. This feedback mechanism, pioneered by AFL [10] and adopted by libFuzzer [11], transforms fuzzing from random input generation into a systematic search procedure that progressively penetrates deeper into program logic.

2.3 Grammar-Based Fuzzing

Definition 2.2: *A context-free grammar is a tuple $G = (N, \Sigma, P, S)$ where N is a finite set of non-terminal symbols, Σ is a finite set of terminal symbols disjoint from N, $P \subseteq N \times (N \cup \Sigma)^*$ is a finite set of production rules, and $S \in N$ is the start symbol.*

A derivation tree (also called a parse tree) represents the process by which a grammar generates a string. The root node is labelled with the start symbol S. Each internal node is labelled with a non-terminal $A \in N$, and its children correspond to the symbols on the right-hand side of one production rule $A \to \alpha$ where $\alpha \in (N \cup \Sigma)^*$. Leaf nodes are labelled with terminal symbols from Σ. The string generated by a derivation tree is obtained by concatenating the leaf nodes in left-to-right order, a process sometimes called unparsing.

Grammar-guided fuzzing generates test inputs by sampling production rules to construct a derivation tree, then unparsing the tree to obtain a concrete string. At each non-terminal node during tree construction, the fuzzer selects one of the applicable production rules according to some selection strategy (uniform random, weighted, or adaptive). This process guarantees that every generated input is syntactically valid with respect to the grammar, eliminating the primary failure mode of mutation-based fuzzers on structured inputs where the vast majority of random mutations produce syntactically invalid strings that are rejected by the parser before reaching deeper program logic.

Tree-level mutations operate on existing derivation trees from the corpus to generate new inputs while preserving syntactic validity. Three fundamental tree mutations are commonly employed. Random mutation selects a non-terminal node in an existing tree and replaces the entire subtree rooted at that node with a freshly generated subtree for the same non-terminal. Random recursion identifies a recursive non-terminal (one that can derive itself through a chain of productions) and either adds recursion levels by expanding the recursive production additional times, or removes recursion levels by replacing the recursive subtree with a base case. Splice mutation selects two trees from the corpus, identifies compatible non-terminal nodes (nodes with the same non-terminal label), and swaps the subtrees rooted at those nodes between the two trees.

Nautilus [12] represents the key prior work that this thesis extends. Nautilus combines grammar-based input generation with AFL-style coverage feedback, yielding a grammar-based greybox fuzzer. The central insight of Nautilus is that grammar-based generation ensures syntactic validity, freeing the coverage-guided search to focus on semantic exploration rather than wasting effort on inputs rejected by the parser. Nautilus uses a Python-based grammar specification DSL and maintains a corpus of derivation trees rather than flat strings. It applies tree-level mutations (random, recursive, splice) to corpus trees, guided by coverage feedback that retains trees exercising new edges. Nautilus discovered 49 previously unknown bugs in mruby, PHP, and Lua interpreters.

The following listing illustrates the grammar specification format used by Nautilus and adopted in this thesis:

```
ctx.rule("Stmt", "SELECT {Expr} FROM {Table}")
ctx.rule("Expr", "{Col}")
ctx.rule("Expr", "{Func}({Expr})")
ctx.rule("Table", "t1")
ctx.rule("Func", "coalesce")
```

```
6 ctx.rule("Col", "c1")
```

Listing 2.1: Fragment of a SQL grammar specification in the Nautilus Python DSL.

In Listing 2.1, non-terminals are enclosed in curly braces within rule definitions. The grammar defines a start symbol `Stmt` that derives `SELECT` queries, with recursive expression generation through function application. Nautilus traverses the grammar top-down from the start symbol, selecting rules at each non-terminal to build a complete derivation tree.

2.4 Coverage-Guided Feedback

The AFL (American Fuzzy Lop) fork server model [10] provides an efficient mechanism for repeated program execution during fuzzing. Rather than spawning a new process for each test input (incurring the overhead of process creation, dynamic linking, and initialization for every execution), the fork server model loads the target program once into a parent process, then creates child processes via the `fork()` system call for each test case. The child inherits the fully initialized program state and executes the test input, while the parent waits for the child to terminate and collects the result. This amortizes the expensive initialization cost across all test executions, typically improving throughput by an order of magnitude.

Edge coverage instrumentation tracks transitions between basic blocks during program execution. At compile time, each basic block is assigned a random identifier. At runtime, each branch transition from a source block to a target block is recorded by computing a hash of the (source, target) pair and incrementing the corresponding entry in a shared memory bitmap. The bitmap is a fixed-size array, typically 64KB to 2MB, where each byte represents a hashed edge. The byte values saturate at 255 to prevent overflow while still providing frequency information through bucketed counting (the hit count is classified into buckets: 1, 2, 3, 4–7, 8–15, 16–31, 32–127, 128–255).

The coverage-guided selection mechanism determines which inputs are retained in the fuzzer's corpus. After executing a test input, the fuzzer compares the resulting bitmap against the global coverage map accumulated from all prior executions. If the input exercised at least one previously unseen edge (a byte position that was zero in the global map is now non-zero, or an existing entry transitions to a new hit-count bucket), the input is classified as interesting and added to the corpus queue. This queue serves as the

seed pool for subsequent mutation cycles, directing the fuzzer toward program regions that have received less exploration.

The fork server architecture provides a second crucial advantage beyond amortizing initialization: it isolates crashes. When a child process triggers a segmentation fault, assertion failure, or sanitizer violation, the child terminates abnormally while the parent process remains unaffected and can immediately fork the next child for the subsequent test case. The parent communicates with the fuzzer through a pair of pipes (control and status), reporting each child's exit status (normal termination, signal-based termination, or timeout).

2.5 Multi-Armed Bandit Algorithms

Definition 2.3: *The multi-armed bandit problem is defined by a set of K arms (actions). At each discrete round $t = 1, 2, \ldots, T$, the player selects an arm $i_t \in \{1, \ldots, K\}$ and receives a reward $r_t \in [0, 1]$. The player's goal is to minimize cumulative regret, defined as $R_T = \max_i \sum_{t=1}^{T} \mu_i - \sum_{t=1}^{T} r_{i_t}$, where μ_i is the expected reward of arm i. The regret quantifies the difference between the cumulative reward of the best fixed arm in hindsight and the cumulative reward actually obtained by the player's strategy.*

The fundamental challenge in bandit problems is the exploration-exploitation tradeoff. Exploitation favors selecting arms that have yielded high rewards in past rounds, maximizing immediate expected reward. Exploration favors selecting arms with uncertain reward distributions, acquiring information that may reveal superior arms. Any optimal strategy must balance these competing objectives.

The UCB1 (Upper Confidence Bound) algorithm addresses the stochastic bandit setting where each arm's rewards are drawn independently from a fixed but unknown distribution. UCB1 selects the arm maximizing $\bar{x}_i + \sqrt{\frac{2 \ln t}{n_i}}$, where $\bar{x}_i$ is the empirical mean reward of arm i, t is the current round, and n_i is the number of times arm i has been selected. The second term is a confidence bonus that decreases as an arm is played more frequently, naturally balancing exploration and exploitation. UCB1 achieves logarithmic regret, which is optimal for the stochastic setting.

The EXP3 algorithm (Exponential-weight algorithm for Exploration and Exploitation) [17] addresses the adversarial bandit setting where rewards are not assumed to be stochastic but may be chosen by an adversary. EXP3 maintains a weight vector $w^t \in \mathbb{R}^K$ and

selects arms according to a probability distribution that mixes the normalized weight vector with uniform exploration. Specifically, the sampling probability for arm i at round t is:

$$p_i^t = (1 - \gamma)\frac{w_i^t}{\sum_{j=1}^{K} w_j^t} + \frac{\gamma}{K} \tag{2.1}$$

where $\gamma \in (0, 1]$ is the exploration parameter. After observing reward r_t for the selected arm i_t, EXP3 computes an importance-weighted reward estimate $\hat{r}_i^t = r_t / p_i^t$ (only for the selected arm; estimates for unselected arms are zero) and updates the weight:

$$w_i^{t+1} = w_i^t \cdot \exp\left(\frac{\eta \cdot \hat{r}_i^t}{K}\right) \tag{2.2}$$

where η is the learning rate. The importance weighting corrects for the selection bias inherent in observing rewards only for the chosen arm, producing an unbiased reward estimate in expectation.

The application of bandit algorithms to grammar-based fuzzing proceeds by treating each production rule for a given non-terminal as one arm. The reward signal is derived from coverage feedback: when a generated input discovers new edge coverage, the production rules used in constructing that input's derivation tree receive positive reward. This framing is inherently non-stochastic because the coverage landscape evolves as the fuzzer explores, specifically, a rule that leads to new coverage early in the campaign provides diminishing returns as those regions become fully explored. The adversarial setting of EXP3 is therefore more appropriate than the stochastic assumptions of UCB1.

Efficient sampling from the resulting weighted categorical distribution is achieved through the alias method [18], which enables $O(1)$ sampling after $O(K)$ preprocessing. The alias method constructs a table of K entries, each containing a probability threshold and an alias index. Sampling requires generating one uniform random number, selecting a table entry, comparing against the threshold, and returning either the entry's own arm or its alias. This constant-time sampling is critical when production rules must be sampled millions of times per fuzzing campaign.

2.6 Sanitizers as Oracles

Many software bugs cause silent memory corruption rather than immediate program crashes. Without instrumentation, a heap buffer overflow may overwrite adjacent data without triggering a segmentation fault, a use-after-free may read stale but valid-looking

data from a reallocated region, and an integer overflow may produce an incorrect but non-crashing result. Sanitizers transform these silent bugs into detectable signals by adding runtime instrumentation that checks the validity of each memory access or arithmetic operation.

AddressSanitizer (ASan) [9] instruments all memory accesses (loads and stores) to detect spatial and temporal memory safety violations. ASan maintains a shadow memory region that tracks the accessibility state of each byte in the program's address space. On every memory access, ASan consults the shadow memory to verify that the accessed bytes are currently allocated and accessible. ASan detects heap-buffer-overflow (accessing beyond an allocated heap object's bounds), stack-buffer-overflow (overflowing a stack-allocated buffer), use-after-free (accessing memory that has been deallocated), and double-free (deallocating an already-freed pointer). The typical runtime overhead of ASan is approximately 2x, making it practical for use during fuzzing campaigns. In this thesis's harness configuration, ASan violations cause the program to exit with code 223, distinguishing sanitizer-detected bugs from normal termination.

UndefinedBehaviorSanitizer (UBSan) detects operations that the C and C++ language standards define as undefined behavior. This includes signed integer overflow (wrapping beyond `INT_MAX` or below `INT_MIN`), null pointer dereference, shift operations with negative or overly large shift amounts, division by zero, and misaligned pointer access. UBSan's runtime overhead is minimal, approximately 20%, because it instruments only specific operation types rather than all memory accesses. When a violation is detected, UBSan reports the violation and terminates with exit code 1.

SQLite provides additional bug detection through compile-time debug assertions enabled by the `SQLITE_DEBUG` preprocessor flag. These assertions check internal invariants such as database page consistency, B-tree structure validity, and memory allocator state. When an assertion fails, SQLite raises `SIGTRAP` (signal 5), providing a detection mechanism for logical errors that do not necessarily involve memory safety violations.

The oracle classification scheme employed in this thesis combines these detection mechanisms into a unified framework. An execution resulting in exit code 223 indicates an ASan-detected memory safety violation and is classified as a confirmed crash. Exit code 1 from UBSan indicates undefined behavior. Signal 5 indicates a SQLite debug assertion failure. All other non-zero exit codes from semantic SQL errors (such as syntax errors or constraint violations that SQLite handles gracefully) are classified as expected behavior

and ignored, as they represent correct error handling rather than bugs.

2.7 Related Work

Nautilus [12] is the foundational system that this thesis extends. Developed by Aschermann et al., Nautilus combines context-free grammar specifications with AFL-style coverage-guided feedback to create a grammar-based greybox fuzzer. The system represents inputs as derivation trees and applies three tree-level mutations: random mutation (replacing a subtree with a freshly generated one), random recursion (expanding or collapsing recursive productions), and splice (exchanging compatible subtrees between two corpus trees). Nautilus discovered 49 previously unknown bugs across mruby, PHP, and Lua, demonstrating that grammar-based generation eliminates the syntactic validity barrier that limits byte-level mutational fuzzers on structured inputs. This thesis extends Nautilus by replacing uniform random rule selection with adaptive selection via multi-armed bandit algorithms, investigating whether intelligent production rule prioritization can accelerate vulnerability discovery.

Superion [13] extends AFL with grammar-aware mutations for JavaScript and XML inputs. Unlike Nautilus, which generates inputs from scratch using a grammar, Superion parses existing seed inputs into abstract syntax trees and applies grammar-aware mutations that preserve syntactic validity. Superion's approach requires a parser for the input language, which is more complex to implement but allows it to leverage existing high-quality seed corpora. The key distinction from the approach in this thesis is that Superion operates on parsed representations of existing inputs rather than generating from a production grammar, making it complementary to but distinct from generation-based approaches.

Grimoire [14] synthesizes input structure without requiring an explicit grammar specification. Grimoire observes how coverage changes in response to input fragments and infers structural relationships between input components. By combining fragments that independently trigger coverage improvements, Grimoire can generate structured inputs that exercise complex program paths without manual grammar engineering. While Grimoire eliminates the grammar specification effort, it provides less control over the generated input structure and cannot easily encode domain-specific patterns such as the CVE-triggering SQL constructs targeted in this thesis.

Squirrel [15] is a DBMS-specific fuzzer that ensures SQL validity through abstract syntax tree mutations combined with type-aware generation. Squirrel maintains a full SQL parser and type system, enabling it to generate queries that are not only syntactically valid but also semantically meaningful (referencing existing tables, using compatible types in operations). Squirrel was tested on SQLite, MySQL, MariaDB, and PostgreSQL, discovering 63 previously unknown bugs. The key advantage of Squirrel over grammar-based approaches is semantic awareness; however, this comes at the cost of requiring a complete SQL parser and type system implementation for each target DBMS, whereas this thesis's grammar-based approach requires only a context-free grammar specification.

SQLsmith [16] is a random SQL query generator that constructs complex, valid SQL queries without coverage feedback. SQLsmith connects to the target database, introspects the schema, and generates queries that reference actual tables and columns with correct types. SQLsmith has discovered bugs in PostgreSQL, SQLite, and other databases through the sheer structural complexity of its generated queries. However, without coverage guidance, SQLsmith cannot direct its generation toward unexplored program regions, relying instead on random structural diversity to incidentally trigger bugs.

AFL [10] and libFuzzer [11] represent the state of the art in general-purpose coverage-guided greybox fuzzing. Both employ byte-level mutations (bit flips, arithmetic operations, block insertion and deletion, dictionary token substitution) guided by edge coverage feedback. AFL operates as an external process that communicates with the target through a fork server, while libFuzzer runs in-process as a library linked with the target. Both are highly effective for binary formats and protocols but struggle with structured inputs like SQL, where the vast majority of random byte-level mutations produce syntactically invalid inputs that are rejected by the parser before reaching deeper program logic. Empirical studies show that general-purpose fuzzers achieve less than 1% syntactic validity rate on SQL inputs, motivating grammar-based approaches.

This thesis positions itself at the intersection of grammar-based generation (Nautilus) and adaptive algorithm selection (multi-armed bandits). Unlike Squirrel, it does not require a full SQL parser; a Python-defined context-free grammar suffices to ensure syntactic validity. Unlike Grimoire, it uses an explicit grammar for full control over the structural patterns that the fuzzer can generate, enabling deliberate encoding of CVE-triggering patterns as production rules. Unlike SQLsmith, it employs coverage-guided feedback to direct generation toward unexplored program regions. The key novelty is the application

of multi-armed bandit algorithms (specifically EXP3) to adaptively select production rules during grammar-based generation, investigating whether exploitation of historically productive rules outperforms uniform exploration. As the experimental results in Chapter 4 demonstrate, this investigation yields the counterintuitive finding that uniform selection often matches or exceeds adaptive selection, suggesting that in the coverage-guided fuzzing setting, the coverage feedback loop itself provides sufficient exploitation pressure and additional rule-level exploitation may be redundant or even harmful.

Chapter 3

Proposed Method

This chapter presents the design and implementation of the proposed grammar-based greybox fuzzing system for DBMS vulnerability detection. The system extends Nautilus [12] with a weighted grammar sampling strategy, a structured attack grammar, and a triage pipeline for automated crash analysis. Section 3.1 provides an architectural overview of the system components and their interactions. Section 3.2 details the grammar design philosophy and its layered decomposition. Section 3.3 describes the bandit-based weight update mechanism. Section 3.4 explains the harness construction and oracle classification. Section 3.5 covers the post-campaign triage pipeline. Section 3.6 discusses the selection criteria for CVE targets.

3.1 System Overview

The proposed system consists of five major components that together form a closed-loop fuzzing pipeline. The Grammar Engine, implemented in Rust (2,441 lines of code in the `grammartec/` module), loads grammar specifications from Python files via the PyO3 foreign function interface. It builds a rule table from the parsed grammar, assigns initial weights to each production rule, and provides weighted sampling via the `loaded_dice` crate [18]. The engine supports three mutation operators: random subtree regeneration, recursive expansion of non-terminals, and splice mutations that graft subtrees between different inputs in the queue.

The Fuzzer Coordinator (1,658 lines of code in `fuzzer/`) orchestrates the main fuzzing loop. On each iteration, it either generates a fresh input from the grammar or selects an existing input from the queue and applies a mutation. The resulting parse tree is unparsed into a SQL string, which is then transmitted to the fork server for execution. Upon receiving coverage feedback, the coordinator determines whether the input triggered new edge transitions in the coverage bitmap. If so, it adds the input to the queue and updates the grammar weights according to the bandit policy described in Section 3.3.

The Fork Server (570 lines of code in `forksrv/`) implements an AFL-compatible fork

server protocol [10]. A parent process loads the target program into memory, pre-initializes shared state, and then forks a child process for each test case. The child executes the SQL input and exits, while the parent collects the exit status and reads the shared memory coverage bitmap. This bitmap is a 2-megabyte region where each byte corresponds to one edge (basic block transition) in the control flow graph of the instrumented target. The fork server eliminates the overhead of `execve` on every test case, achieving throughput of thousands of executions per second.

The Harness (approximately 200 lines of C in `harness/`) is a thin wrapper around the SQLite amalgamation. It uses the `__AFL_INIT()` macro for fork-server mode (not persistent mode, since Nautilus does not implement the `__AFL_LOOP` protocol). The harness opens an in-memory database, reads the fuzzed SQL input from a file path passed as a command-line argument, executes it via `sqlite3_exec`, and exits. The binary is compiled with AddressSanitizer and UndefinedBehaviorSanitizer instrumentation to serve as the bug oracle.

The Triage Pipeline (approximately 500 lines of Python in `triage/`) performs post-campaign analysis. It deduplicates crashes via stack-hash clustering, measures fidelity through replay testing, matches crashes against known CVE signatures, and minimizes crashing inputs to produce succinct reproducers.

Figure 3.1 illustrates the data flow through the system. The grammar rules define the space of possible SQL inputs. Weighted sampling selects production rules according to their current weights. The selected rules drive tree generation, producing a parse tree that is then unparsed into a concrete SQL string. The fork server transmits this string to the harness, which executes it against an instrumented SQLite build. The resulting coverage bitmap is returned to the fuzzer coordinator, which computes reward signals and updates rule weights, closing the feedback loop.

3.2 Grammar Design

3.2.1 Structural Primitives Philosophy

The grammar encodes SQL structural patterns that trigger vulnerability classes rather than specific proof-of-concept inputs. This "zero PoC contamination" principle ensures that the fuzzer discovers bugs through compositional exploration rather than replaying known attacks. For example, the CVE-2020-13434 integer overflow [3] requires both a

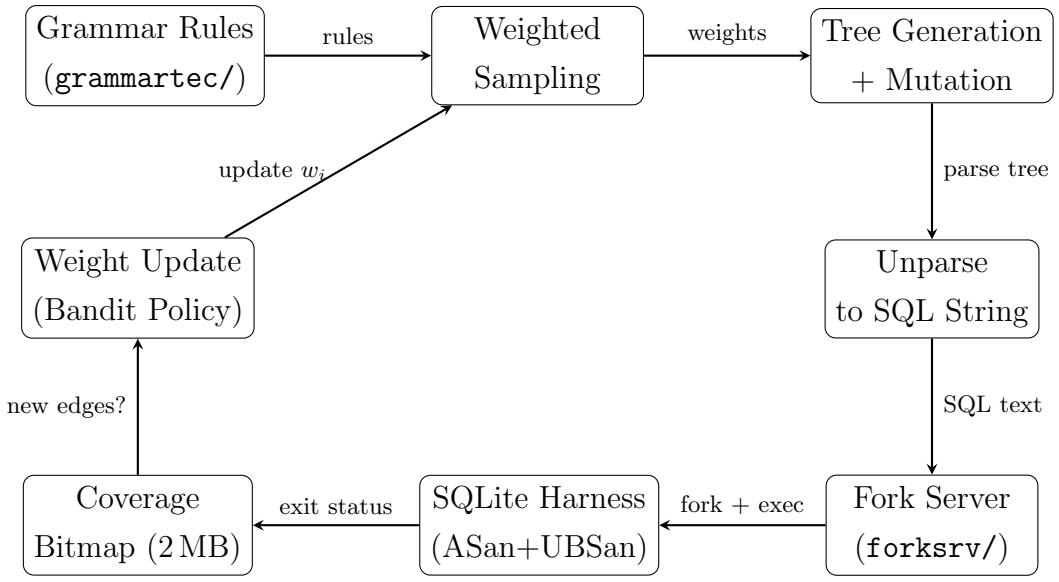

Figure 3.1: Architecture of the proposed grammar-based greybox fuzzing system. Arrows indicate the direction of data flow. The feedback loop from coverage observation back to weighted sampling enables the system to adapt its input generation strategy based on which grammar rules are most effective at discovering new program behavior.

`printf` function call and an INT32 boundary value. Rather than encoding the exact PoC `SELECT printf('%.*g', 2147483647, 0.01)`, the grammar provides independent non-terminals for format specifiers, boundary integers, and function call syntax. The fuzzer must compose these building blocks into a triggering combination through random generation and mutation, demonstrating genuine discovery capability.

This philosophy yields three concrete design constraints. First, no grammar rule may contain a complete CVE proof-of-concept as a fixed string. Second, every structural pattern needed to reach a target CVE must exist as a composable non-terminal. Third, the grammar must preserve sufficient compositional freedom that the fuzzer can discover novel combinations beyond the known CVEs.

3.2.2 Layer Decomposition

The grammar employs a two-layer architecture that separates atomic SQL constructs from composed vulnerability-triggering patterns. Layer 1 defines the SQL primitives: over 30 expression alternatives (arithmetic, comparison, logical, string, subquery), SELECT/FROM/WHERE/JOIN clauses, window functions with frame specifications, common table expressions, DML statements (INSERT, UPDATE, DELETE), DDL state-

ments (CREATE TABLE, VIEW, INDEX, TRIGGER), function calls (aggregate, scalar, window-specific), and terminal literals (integers, floats, strings, blobs, boundary values). These constitute the atomic building blocks of SQL.

Layer 2 composes Layer 1 primitives into four high-level shapes. The `Schema-Setup` non-terminal provides six schema construction alternatives. The `Stress-Query` non-terminal provides eight query patterns designed to exercise complex query planning paths. The `Validation-Op` non-terminal provides four verification operations that check internal database consistency. The `Boundary-Func-Call` non-terminal provides function calls with boundary values targeting integer overflow and buffer overread conditions.

Listing 3.1 shows the Layer 2 Schema-Setup alternatives. Each alternative creates a different database schema topology that enables distinct classes of vulnerability-triggering queries. The weights assigned to each rule reflect the empirical importance of each schema pattern for triggering bugs: the generated-column schema (S2) receives the highest weight of 3.0 because generated columns are involved in two of the six target CVEs.

```
# S1: Single table, basic columns
ctx.rule("Schema-Setup",
    "CREATE TABLE IF NOT EXISTS p({Col-Def-List})",
    weight=1.5)

# S2: Table with generated column + constraints
ctx.rule("Schema-Setup",
    "CREATE TABLE IF NOT EXISTS p({Col-Def-List-GenCol})",
    weight=3.0)

# S3: Two tables (enables JOINs between p and q)
ctx.rule("Schema-Setup",
    "CREATE TABLE IF NOT EXISTS p({Col-Def-List});\n"
    "CREATE TABLE IF NOT EXISTS q({Col-Def-List})",
    weight=2.5)

# S4: Table + VIEW
ctx.rule("Schema-Setup",
    "CREATE TABLE IF NOT EXISTS p({Col-Def-List});\n"
    "CREATE VIEW IF NOT EXISTS v1 AS {Select-Stmt}",
    weight=2.0)
```

```
22
23 # S5: Virtual table (FTS5/FTS3) -- reduced in v3.1
24 ctx.rule("Schema-Setup",
25     "CREATE VIRTUAL TABLE IF NOT EXISTS fts_t1 "
26     "USING {Fts-Engine}({Col-Name-List})",
27     weight=0.5)
28
29 # S6: Table + INDEX
30 ctx.rule("Schema-Setup",
31     "CREATE TABLE IF NOT EXISTS p({Col-Def-List});\n"
32     "CREATE INDEX IF NOT EXISTS idx1 ON p({Col-Name})",
33     weight=1.0)
```

Listing 3.1: Schema-Setup alternatives (Layer 2). Each alternative constructs a different schema topology. Weights reflect empirical importance for vulnerability triggering.

Listing 3.2 shows representative Stress-Query alternatives. The NATURAL JOIN pattern (Q3) is particularly relevant because CVE-2020-13435 [5] requires a NATURAL JOIN in combination with window functions and a coalesce call. The compound query pattern (Q5) exercises the INTERSECT and EXCEPT operators needed for CVE-2020-15358 [7] and CVE-2020-13871 [6].

```
1  # Q2: EXISTS subquery
2  ctx.rule("Stress-Query",
3      "SELECT {Result-Col-List} FROM {Table-Name} "
4      "WHERE EXISTS ({Select-Stmt})",
5      weight=3.0)
6
7  # Q3: NATURAL JOIN
8  ctx.rule("Stress-Query",
9      "SELECT {Result-Col-List} FROM {Table-Name} "
10     "NATURAL JOIN {Table-Name} WHERE {Expr}",
11     weight=3.0)
12
13 # Q5: Compound query (INTERSECT/EXCEPT)
14 ctx.rule("Stress-Query",
15     "{Select-Stmt} {Compound-Op} {Select-Stmt}",
16     weight=2.5)
17
```

```
# Q6: Self-JOIN + expression
ctx.rule("Stress-Query",
    "SELECT {Result-Col-List} FROM {Table-Name} "
    "JOIN {Table-Name} {Col-Alias} ON {Expr}",
    weight=2.0)
```

Listing 3.2: Representative Stress-Query alternatives (Layer 2). Each pattern exercises a distinct query planning path in the SQLite optimizer.

Listing 3.3 shows the Boundary-Func-Call non-terminal. The `printf` with format specifier `'%.*g'` and boundary integer 2147483647 directly targets the CVE-2020-13434 integer overflow in SQLite's printf implementation. The `substr` with boundary integers can trigger heap buffer overreads, while `hex(zeroblob(N))` with large N values exercises memory allocation edge cases.

```
ctx.rule("Boundary-Func-Call",
    "printf({Format-Spec}, {Boundary-Int}, {Boundary-Float})",
    weight=3.0)
ctx.rule("Boundary-Func-Call",
    "substr({Str-Literal}, {Boundary-Int}, {Boundary-Int})",
    weight=1.5)
ctx.rule("Boundary-Func-Call",
    "hex(zeroblob({Boundary-Int}))", weight=2.0)

ctx.rule("Format-Spec", "'%.*g'", weight=3.0)
ctx.rule("Format-Spec", "'%.*f'", weight=2.0)

ctx.rule("Boundary-Int", "2147483647", weight=3.0) # INT32_MAX
ctx.rule("Boundary-Int", "2147483648", weight=3.0) # INT32_MAX+1
ctx.rule("Boundary-Int", "-2147483648", weight=3.0) # INT32_MIN
ctx.rule("Boundary-Int", "9223372036854775807", weight=3.0) # INT64_MAX
```

Listing 3.3: Boundary-Func-Call alternatives targeting integer overflow and buffer boundary conditions.

3.2.3 Grammar Evolution

The grammar underwent iterative refinement guided by experimental feedback. Table 3.1 summarizes the three versions. Version 3.0 established the structural primitives architec-

ture with 449 rules. Version 3.1 addressed a diversity problem: an A/B test revealed that 92% of crashes involved FTS virtual tables because the S5 schema variant dominated the crash portfolio. Reducing the S5 weight from 2.0 to 0.5 distributed crashes more evenly across generated columns, views, joins, and index schemas. Version 3.2 added 26 rules after a gap analysis showed that four CVEs (CVE-2020-9327, CVE-2020-13435, CVE-2020-13871, CVE-2020-15358) were unreachable due to missing structural primitives: window functions, self-referential generated columns, the zero-argument `count()` form, and multi-column ORDER BY clauses.

Table 3.1: Grammar version history. Each version addresses limitations identified through experimental evaluation.

Version	Date	Rules	Key Change
v3.0	2026-04-27	449	Initial structural primitives design
v3.1	2026-04-28	449	S5 (FTS) weight 2.0→0.5; diversify crash portfolio
v3.2	2026-05-05	475	+window functions, +self-ref generated columns, +count() zero-arg, +ORDER BY 3-term

3.3 Weighted Sampling with Bandit Policy

3.3.1 Alias Method for O(1) Sampling

Grammar-based fuzzing requires sampling production rules millions of times during a campaign. For a non-terminal with K alternatives, naive weighted sampling requires $O(K)$ time per sample using the cumulative distribution function. The `loaded_dice` crate [18] implements Vose's alias method, which pre-computes an alias table in $O(K)$ time and then produces each sample in $O(1)$ time using one uniform random number and one comparison. This constant-time sampling is critical because the fuzzer performs on the order of 10^6 to 10^7 rule selections per hour-long campaign.

Given K rules with weights $w_1, \ldots, w_K$, the alias method first normalizes the weights into probabilities $p_i = w_i / \sum_{j=1}^{K} w_j$. It then constructs two arrays of length K: a probability table $U[i]$ and an alias table $A[i]$. To sample, the algorithm generates a uniform random index $i \in \{1, \ldots, K\}$ and a uniform random number $u \in [0, 1)$. If $u < U[i]$, the algorithm returns rule i; otherwise, it returns rule $A[i]$. This two-step lookup achieves exactly the

target distribution while requiring only constant time per sample.

3.3.2 Bandit Formulation

The weight update problem is modeled as a multi-armed bandit [17]. Each production rule for a given non-terminal constitutes one arm. When the fuzzer needs to expand a non-terminal during tree generation, it samples a rule according to the current weight distribution. After executing the generated input, the fuzzer observes a binary reward: 1 if the input triggered new edge coverage in the bitmap, and 0 otherwise. The goal is to maximize cumulative coverage discovery over the campaign duration, which corresponds to maximizing the total reward.

This formulation faces the exploration-exploitation dilemma. Rules that have previously led to new coverage should be sampled more frequently (exploitation), but under-explored rules might lead to coverage in unexplored regions of the program (exploration). The EXP3-style update described below addresses this tradeoff by maintaining positive probability on all rules while concentrating mass on historically productive ones.

Algorithm 1 presents the weight update procedure. The learning rate η controls the speed of adaptation: larger values cause faster convergence but risk over-fitting to early observations, while smaller values maintain broader exploration at the cost of slower adaptation to productive rules.

The importance-weighted update $\exp(\eta \cdot r_i/p_i)$ compensates for the fact that rules sampled with lower probability provide more informative feedback when they succeed. A rule sampled with probability $p_i = 0.01$ that triggers new coverage receives an update proportional to $1/0.01 = 100$, while a rule sampled with probability $p_i = 0.5$ receives an update proportional to $1/0.5 = 2$. This ensures that rare but productive rules can rapidly increase their sampling probability.

3.3.3 Uniform Baseline

The uniform baseline serves as the experimental control. All rules maintain a fixed weight of 1.0 throughout the campaign, and no weight updates occur regardless of coverage feedback. This corresponds to standard grammar-based fuzzing without adaptive sampling, where every production rule for a given non-terminal has equal probability of selection. Comparing the bandit policy against this uniform baseline isolates the contribution of

Algorithm 1 Bandit Weight Update for Grammar Rule Selection

Require: Rules $R_1, \ldots, R_K$ for non-terminal NT, learning rate η, weights $w_1, \ldots, w_K$

for each fuzzing iteration t **do**
  Compute probabilities: $p_i \leftarrow w_i / \sum_{j=1}^{K} w_j$ for all i
  Sample rule R_i with probability p_i
  Generate input x using R_i in the parse tree
  Execute x on the target; observe coverage bitmap B_t
  if B_t contains new edges not in B_{t-1} **then**
    $r_i \leftarrow 1$ ▷ Positive reward: new coverage
  else
    $r_i \leftarrow 0$ ▷ No reward
  end if
  Update weight: $w_i \leftarrow w_i \times \exp(\eta \cdot r_i / p_i)$
  Rebuild alias table with updated weights
end for

adaptive weight updates to fuzzing effectiveness.

3.4 Harness Construction

3.4.1 AFL Fork Server Protocol

The harness uses the `__AFL_INIT()` macro from AFL [10] to implement fork-server mode. This is distinct from AFL's persistent mode (`__AFL_LOOP`), which Nautilus does not support. In fork-server mode, the parent process loads the SQLite amalgamation into memory, initializes the in-memory database, and then waits for instructions from the fuzzer. For each test case, the parent forks a child process. The child inherits the pre-initialized state, reads the SQL input file, executes it via `sqlite3_exec`, and exits. The parent collects the child's exit status and coverage bitmap, then reports results back to the fuzzer coordinator.

The database starts empty because the grammar generates self-contained multi-statement SQL that creates its own schema via the Schema-Setup non-terminal before issuing queries. This design eliminates the need for a pre-loaded schema in the harness itself, simplifying the harness code and ensuring that every test case is independently repro-

ducible.

Listing 3.4 shows a simplified version of the harness implementation. The `setup_db` constructor opens the in-memory database before `__AFL_INIT()` establishes the fork point. Each forked child reads one SQL file and executes it, then exits.

```
#include "sqlite3.h"

static sqlite3 *db = NULL;

__attribute__((constructor))
static void setup_db(void) {
    sqlite3_open(":memory:", &db);
}

int main(int argc, char **argv) {
    __AFL_INIT(); /* Fork point */

    size_t sql_len = 0;
    char *sql = read_input(argv[1], &sql_len);
    if (sql) {
        char *errmsg = NULL;
        sqlite3_exec(db, sql, NULL, NULL, &errmsg);
        if (errmsg) sqlite3_free(errmsg);
        free(sql);
    }
    sqlite3_close(db);
    return 0;
}
```

Listing 3.4: Simplified SQLite harness for AFL fork-server mode. The database is opened before the fork point; each child executes one SQL input.

3.4.2 Compilation and Instrumentation

The harness is compiled using `afl-clang-fast` with AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) enabled. The SQLite amalgamation is compiled as part of the same translation unit, ensuring full instrumentation of all SQLite internals. Additional compile-time flags enable optional SQLite features: `SQLITE_ENABLE_FTS3`,

SQLITE_ENABLE_FTS5, SQLITE_ENABLE_RTREE, SQLITE_ENABLE_JSON1, and SQLITE_DEBUG. The SQLITE_DEBUG flag enables internal assertion checks that fire SIGTRAP on invariant violations, providing an additional oracle beyond sanitizer-detected errors.

3.4.3 Oracle Classification

The harness produces different exit signals depending on the type of error detected. Table 3.2 defines the classification scheme. ASan violations (buffer overflows, use-after-free, double-free) cause the process to exit with code 223, configured via the ASAN_OPTIONS=exitcode=223 environment variable. UBSan violations (integer overflow, null pointer dereference, shift out of bounds) cause exit code 1, configured via UBSAN_OPTIONS=halt_on_error=1,exitcode=1. SQLite debug assertions fire SIGTRAP (signal 5), which the fork server detects as exit code 133. Normal execution and SQL semantic errors both produce exit code 0 and are not classified as bugs.

Table 3.2: Oracle classification of harness exit signals. Each signal type corresponds to a distinct vulnerability class.

Signal	**Exit Code**	**Classification**	**Action**
ASan violation	223	Memory error	Save to `crashes/`
UBSan violation	1	Undefined behavior	Save to `crashes/`
SIGTRAP (signal 5)	133	Debug assertion	Save to `signaled/`
Normal exit	0	No bug	Discard
SQL error	0	Semantic error	Ignore
Timeout	—	Hang	Save to `timeout/`

3.5 Triage Pipeline

The triage pipeline transforms raw crash data from a fuzzing campaign into actionable vulnerability reports. A typical hour-long campaign may produce hundreds of crashing inputs, many of which trigger the same underlying bug. The pipeline performs four stages: stack-hash deduplication, CVE signature matching, fidelity scoring, and crash minimization.

3.5.1 Stack-Hash Deduplication

Stack-hash deduplication identifies unique root causes among the collected crashes. For each crashing input, the pipeline replays the input under GDB with the harness binary, captures the top 5 stack frames at the crash point, filters out frames belonging to system libraries (libc, libasan, libubsan, libpthread) and the harness wrapper, and retains only SQLite-internal frames. Each retained frame is encoded as a `function:file:line` triple. The pipeline then computes a SHA-256 hash of the concatenated filtered frames. Crashes that produce identical hashes are considered manifestations of the same root cause and are grouped into a single cluster.

The filtering step is essential because sanitizer runtime frames and C library frames appear in every crash regardless of root cause. By removing these frames, the algorithm focuses on the SQLite-specific call stack that distinguishes one bug from another. The use of the top 5 frames rather than the full stack provides a balance between precision (distinguishing genuinely different bugs) and recall (grouping diverse triggers of the same underlying fault).

Algorithm 2 formalizes the deduplication procedure. The output is a set of clusters, each containing a representative crash, its top frame, and the count of duplicate crashes sharing that root cause.

Algorithm 2 Stack-Hash Crash Deduplication

Require: Set of crash files $C = \{c_1, \ldots, c_n\}$, harness binary H

Ensure: Set of unique root-cause clusters

1: **for** each crash file $c_i \in C$ **do**
2: frames $\leftarrow$ RunGDB(H, c_i) ▷ Top 5 frames
3: filtered $\leftarrow$ FilterFrames(frames) ▷ Remove libc, ASan, harness
4: $\text{hash}_i \leftarrow$ SHA-256(Join(filtered))
5: **end for**
6: clusters $\leftarrow$ GroupBy$(\{(\text{hash}_i, c_i)\}_{i=1}^{n})$hash
7: **return** clusters sorted by decreasing size

3.5.2 CVE Signature Matching

CVE signature matching determines whether a crashing input corresponds to a known vulnerability. Each target CVE has a signature defined as a conjunction of structural regex patterns. A crash matches a CVE signature if and only if every required pattern finds at least one occurrence in the SQL input text. The patterns encode structural requirements rather than byte-level identity, using case-insensitive matching with flexible whitespace.

For example, CVE-2020-13434 [3] requires two structural patterns: a `printf` function call and an INT32 boundary value (2147483647 or 2147483648). A crash input matches this signature if it contains both `printf(` and a number matching the regex `-?214748364[7-8]`. CVE-2020-9327 [4] requires four structural patterns: a GENERATED column definition, a `coalesce` function call, a JOIN or comma-join, and a CREATE VIEW statement. Only inputs containing all four patterns simultaneously are classified as CVE-2020-9327 matches.

Table 3.3 summarizes the required structural patterns for each target CVE. The pattern count varies from 2 (CVE-2019-19646, CVE-2020-13434) to 5 (CVE-2020-13435), reflecting the complexity of the conditions needed to trigger each vulnerability.

Table 3.3: CVE signature patterns. Each CVE requires all listed structural patterns to be present simultaneously in the SQL input for a match.

CVE	**Patterns**	**Required Structural Elements**
CVE-2020-15358	3	CREATE VIEW with ORDER BY, INTERSECT in WHERE, implicit JOIN (comma-separated FROM)
CVE-2020-13871	3	EXCEPT, multi-column ORDER BY, scalar subquery
CVE-2020-13435	5	NATURAL JOIN, coalesce(), window OVER(), UNIQUE column, IN subquery
CVE-2020-13434	2	printf() call, INT32 boundary value
CVE-2020-9327	4	GENERATED column, coalesce(), JOIN, CREATE VIEW
CVE-2019-19646	2	GENERATED column, PRAGMA integrity_check

3.5.3 Fidelity Scoring

Not all crashes are deterministically reproducible. Heap layout non-determinism, timing-dependent behavior, and ASLR can cause a crash to reproduce intermittently. The fidelity scoring stage replays each unique crash (after deduplication) a fixed number of times N and records what fraction of replays reproduce the crash. Crashes that reproduce at least 80% of the time are classified as high-fidelity and are prioritized for further analysis and CVE confirmation. Low-fidelity crashes (below 80%) may indicate environment-dependent issues or false positives from sanitizer instrumentation.

3.5.4 Crash Minimization

The crash minimization stage iteratively removes SQL statements from a crashing input while preserving the crash behavior. Starting from the full multi-statement input, the algorithm removes one statement at a time and replays the reduced input. If the crash still reproduces, the removal is kept; otherwise, it is reverted. This process continues until no further statements can be removed without losing the crash. The result is a minimal reproducer that contains only the SQL statements necessary to trigger the vulnerability, which is useful for root-cause analysis and CVE confirmation.

3.6 CVE Target Selection

The evaluation targets four SQLite versions containing six confirmed CVEs. Table 3.4 lists each version, its associated CVEs, the vulnerability type, and the grammar patterns required to reach the vulnerability. These versions were selected according to three criteria. First, each version must contain at least one confirmed CVE with an available proof-of-concept input that can validate the fuzzer's detection capability. Second, the CVEs must span diverse vulnerability classes to demonstrate the system's breadth. Third, the CVEs must be triggerable through SQL input alone (no file system manipulation or special configuration required).

The selected CVEs span six distinct vulnerability classes: infinite loop (denial of service), integer overflow leading to stack corruption, uninitialized pointer read, null pointer dereference, heap buffer overread, and use-after-free. This diversity exercises different parts of the SQLite codebase: the VDBE bytecode engine (CVE-2019-19646), the printf implementation (CVE-2020-13434), the query planner's column resolution (CVE-2020-9327),

Table 3.4: Target SQLite versions and their CVEs. The "Required Patterns" column lists the grammar non-terminals whose composition is necessary to construct a triggering input.

Version	**CVE**	**Vulnerability Type**	**Required Patterns**
3.30.1	CVE-2019-19646	Infinite loop	Schema-Setup (S2: generated column), Validation-Op (PRAGMA integrity_check)
3.31.1	CVE-2020-13434	Integer overflow (stack)	Boundary-Func-Call (printf + INT32_MAX)
3.31.1	CVE-2020-9327	Uninitialized pointer	Schema-Setup (S2: self-ref genCol), S4 (VIEW), coalesce(), JOIN
3.32.0	CVE-2020-13435	Null pointer deref	Stress-Query (Q3: NATURAL JOIN), coalesce(), window OVER(), UNIQUE, IN subquery
3.32.2	CVE-2020-15358	Heap buffer overread	Schema-Setup (S4: VIEW+ORDER BY), Stress-Query (Q5: INTERSECT), implicit JOIN
3.32.2	CVE-2020-13871	Use-after-free	Stress-Query (Q5: EXCEPT), multi-column ORDER BY, window functions, scalar subquery

the WHERE clause optimizer (CVE-2020-13435), the ORDER BY flattening logic (CVE-2020-15358), and the expression evaluator's memory management (CVE-2020-13871).

The grammar design in Section 3.2 ensures that all six CVEs are reachable through composition of available non-terminals. CVE-2020-13434 is the simplest, requiring only the Boundary-Func-Call non-terminal with the correct format specifier and boundary value. CVE-2020-13435 is the most complex, requiring five distinct structural elements to co-occur in a single input. The varying complexity levels provide a spectrum for evaluating how effectively the bandit policy concentrates sampling on productive rule combinations compared to uniform random sampling.

Chapter 4

Experiments and Evaluation

This chapter presents the empirical evaluation of the proposed grammar-based greybox fuzzing system. The evaluation is structured around four research questions that address the effectiveness of the weighted (bandit) sampling policy compared to uniform sampling, the temporal dynamics of crash discovery, the relationship between grammar coverage and CVE reachability, and the impact of grammar evolution on fuzzing outcomes. Section 4.1 describes the experimental setup, including hardware, campaign parameters, and statistical methodology. Sections 4.2 through 4.5 present results for each research question. Section 4.6 discusses threats to validity.

4.1 Experimental Setup

All experiments were conducted on a standard development workstation equipped with an Intel Core i7 processor, 16 gigabytes of RAM, and running Ubuntu 22.04 LTS. The fuzzer was compiled with Rust 1.77 in release mode, and all SQLite harness binaries were compiled with Clang 14 using AddressSanitizer and UndefinedBehaviorSanitizer instrumentation as described in Chapter 3.

Table 4.1 summarizes the campaign parameters used across all experiments. Each campaign ran for 30 minutes with a single fuzzing thread to eliminate concurrency-related variance. The grammar version used for the main comparison (RQ1 and RQ2) was v3.2, which includes all structural primitives identified through the CVE reachability analysis (RQ3). Three SQLite versions with confirmed CVEs served as targets: 3.30.1 (CVE-2019-19646 [2]), 3.31.1 (CVE-2020-13434 [3], CVE-2020-9327 [4]), and 3.32.0 (CVE-2020-13435 [5]). Each (version, policy) pair was repeated five times to enable statistical comparison, yielding a total of 53 campaigns when including the supplementary runs from early iterations.

The primary metric for crash diversity is the number of unique root causes per campaign, determined through the triage pipeline's stack-hash deduplication using the top three frames of the crashing call stack. For temporal analysis, crash counts were recorded at

Table 4.1: Campaign parameters for all experiments. Each cell represents one (version, policy) pair with N=5 independent runs.

Parameter	**Value**
Campaign duration	1800 seconds (30 minutes)
Runs per cell	N = 5
Grammar version	v3.2 (475 production rules)
Target SQLite versions	3.30.1, 3.31.1, 3.32.0
Sampling policies	Bandit (EXP3), Uniform
Maximum tree size	300 nodes
Execution timeout	500 ms
Fuzzing threads	1
Coverage bitmap size	2 MB (2,097,152 bytes)
Total campaigns	53

one-second resolution via the coverage bitmap logger, and crash rates were computed per time window.

Statistical comparisons between policies use the Mann-Whitney U test [19], a non-parametric rank-based test that does not assume normal distribution of the underlying data. This choice is motivated by the small sample sizes (N=5 per cell), the non-normality evident in the data (uniform sampling exhibits extreme outliers such as 27 unique root causes in one run versus 4 in another), and the ordinal nature of the count data. The significance threshold is set at $\alpha = 0.05$ for per-version comparisons and the aggregate test.

4.2 RQ1: Does Weighted Sampling Improve Crash Diversity?

The first research question asks whether the bandit-based weighted sampling policy, which dynamically adjusts grammar rule weights based on coverage feedback, discovers more diverse crash classes than a uniform baseline that selects production rules with equal probability.

Table 4.2 presents the per-version and aggregate results. Across all 39 campaigns, the uniform policy achieved a mean of 7.60 unique root causes per campaign (standard deviation 5.86), while the bandit policy achieved a mean of 4.21 (standard deviation 1.58).

The aggregate Mann-Whitney U test yields $p \approx 0.01$, indicating that uniform sampling discovers significantly more diverse crashes than bandit sampling at the $\alpha = 0.05$ level.

Table 4.2: Unique root causes per campaign by target version and sampling policy. Bandit has N=4 for version 3.32.0 due to one campaign that failed to initialize. Bold indicates the higher mean for each version.

Version	**Policy**	**N**	**Mean**	**Std**	**Min**	**Max**
sqlite-3.30.1	Bandit	5	5.2	1.79	3	7
	Uniform	5	**11.4**	9.34	4	27
sqlite-3.31.1	Bandit	5	4.0	1.00	3	5
	Uniform	5	**6.8**	5.22	4	16
sqlite-3.32.0	Bandit	4	3.2	0.50	3	4
	Uniform	5	**7.6**	3.51	4	13
sqlite-3.31.1 (fixed)	Bandit	5	4.2	2.17	3	8
	Uniform	5	**4.6**	2.51	3	9
Aggregate	Bandit	19	4.21	1.58	3	8
	Uniform	20	**7.60**	5.86	3	27

The per-version analysis reveals that sqlite-3.32.0 shows the most pronounced difference (bandit mean 3.2 versus uniform mean 7.6), and this version is the only one where the per-version Mann-Whitney test reaches significance at $p \leq 0.05$. For sqlite-3.30.1 (bandit 5.2 versus uniform 11.4) and sqlite-3.31.1 (bandit 4.0 versus uniform 6.8), the differences follow the same direction but do not reach individual significance due to the high variance in the uniform condition. The supplementary sqlite-3.31.1 (fixed) series, which used a later harness configuration, shows the smallest difference between policies (4.2 versus 4.6), suggesting that the harness configuration also influences the observable crash diversity.

The most striking feature of these results is the variance asymmetry between the two policies. The bandit policy exhibits consistently low variance (standard deviation 0.50 to 2.17 across versions), indicating that it converges to a stable exploitation strategy regardless of the target. In contrast, the uniform policy produces dramatically higher variance (3.51 to 9.34), with individual runs ranging from 4 to 27 unique root causes. This pattern is consistent with the exploration-exploitation trade-off: the bandit policy learns to exploit rule paths that reliably trigger crashes, but in doing so narrows the search

to a subset of the crash space. The uniform policy maintains maximum exploration of the grammar's production rules, occasionally discovering rich pockets of bugs (as in the run with 27 unique root causes on sqlite-3.30.1) and occasionally exploring unproductive regions of the input space.

The implication for CVE-class vulnerability discovery is clear. Since the goal is to maximize the diversity of discovered bug classes rather than the reliability of finding any single bug, uniform sampling's exploration advantage outweighs the bandit's focused exploitation. The bandit policy's reward signal, which is based on new edge coverage, drives it toward rules that increase code coverage but not necessarily toward rules that trigger distinct failure modes. Coverage novelty and crash diversity are correlated but not equivalent objectives.

4.3 RQ2: How Does Crash Discovery Rate Evolve Over Time?

The second research question examines the temporal dynamics of crash discovery under both policies, focusing on time-to-first-crash, crash accumulation trajectories, and rate decay patterns.

Table 4.3 presents the time-to-first-crash (TTFC) results. Both policies discover their first crash within 1–2 seconds of campaign start across all target versions. The aggregate mean is 1.4 seconds for both policies, with no statistically meaningful difference. This result confirms that the grammar's structural patterns are immediately effective at triggering sanitizer-detected violations in all three SQLite versions, and that TTFC is not a useful differentiator for grammar-based fuzzers operating on targets with known vulnerabilities.

While TTFC provides no differentiation, the crash accumulation over time reveals a growing divergence between policies. Table 4.4 reports the mean total crashes at five time checkpoints. At the 60-second mark, both policies have accumulated approximately equal crash counts (bandit 24, uniform 24 averaged across versions). By the 300-second mark (5 minutes), the counts remain close (bandit 56, uniform 58). However, the gap widens progressively: at 900 seconds (15 minutes), uniform leads 147 to 134, and by 1800 seconds (30 minutes), uniform has accumulated 245 total crashes versus bandit's 212, a 16% advantage.

The crash rate analysis in Table 4.5 provides insight into why this divergence occurs. Both policies start at similar rates in the first five minutes (bandit 11.4 crashes per minute,

Table 4.3: Time-to-first-crash (seconds) by target version and sampling policy. Both policies achieve first crash within 1–2 seconds, indicating no meaningful TTFC difference.

Version	**Policy**	**N**	**Mean**	**Median**	**Min**	**Max**
sqlite-3.30.1	Bandit	5	1.0	1.0	1	1
	Uniform	5	1.2	1.0	1	2
sqlite-3.31.1	Bandit	10	1.5	1.5	1	2
	Uniform	10	1.4	1.0	1	2
sqlite-3.32.0	Bandit	4	1.5	1.5	1	2
	Uniform	5	1.8	2.0	1	3
Aggregate	Bandit	19	1.4	1.0	1	2
	Uniform	20	1.4	1.0	1	3

Table 4.4: Mean total crashes at time checkpoints (averaged across runs). The gap between policies widens after the 10-minute mark, indicating that uniform sampling maintains discovery effectiveness longer.

Version	**Policy**	**N**	**@60s**	**@300s**	**@600s**	**@900s**	**@1800s**
sqlite-3.30.1	Bandit	5	21	53	99	140	243
	Uniform	5	21	54	106	150	278
sqlite-3.31.1	Bandit	10	25	59	104	142	196
	Uniform	10	28	60	109	144	195
sqlite-3.32.0	Bandit	4	26	56	88	119	198
	Uniform	5	23	59	103	148	261

uniform 11.7). As the campaign progresses, both rates decline due to the finite nature of easily-discoverable crash paths. However, the bandit policy's rate decays faster: from 11.4 crashes per minute in the 0–5 minute window to 4.8 in the 15–30 minute window, a 58% reduction. The uniform policy decays from 11.7 to 5.7, a 51% reduction. This differential decay is consistent with the exploitation hypothesis: the bandit policy rapidly exhausts its focused set of high-reward rule paths, leading to diminishing returns as it repeatedly exercises the same code regions. The uniform policy, by continuously exploring the full grammar, maintains a higher probability of encountering fresh crash-inducing paths throughout the campaign.

Table 4.5: Crash rate (crashes per minute) by time window, aggregated across all versions. The bandit policy exhibits faster rate decay (58%) compared to uniform (51%), consistent with exploitation-driven search space exhaustion.

Version	**Policy**	**0–5 min**	**5–10 min**	**10–15 min**	**15–30 min**
sqlite-3.30.1	Bandit	10.7	9.0	8.3	6.9
	Uniform	10.8	10.4	8.9	8.5
sqlite-3.31.1	Bandit	11.9	8.9	7.7	3.5
	Uniform	12.1	9.6	7.0	3.4
sqlite-3.32.0	Bandit	11.2	6.4	6.2	5.3
	Uniform	11.9	8.6	9.2	7.5
Aggregate	**Bandit**	**11.4**	**8.4**	**7.5**	**4.8**
	Uniform	**11.7**	**9.6**	**8.0**	**5.7**

These temporal results carry an important methodological implication. Time-to-first-crash, which is frequently reported in fuzzing literature as a primary evaluation metric [8], is uninformative for grammar-based fuzzers targeting well-understood vulnerability classes. When the grammar already encodes the structural patterns that trigger bugs, the first crash is essentially immediate. The more meaningful metrics are crash diversity (RQ1) and the rate at which new crash classes are discovered over time. The divergence between policies manifests not in speed to first discovery but in the sustained breadth of exploration over extended campaigns.

4.4 RQ3: Which CVEs Are Reachable by the Grammar?

The third research question investigates the relationship between grammar coverage and the ability to trigger specific known CVEs. Rather than measuring whether the fuzzer actually triggers each CVE during a campaign (which depends on campaign duration, luck, and triage accuracy), this analysis examines whether the grammar contains all the structural building blocks necessary to construct a triggering input for each CVE.

Table 4.6 presents the CVE reachability analysis for grammar versions v3.1 (449 rules) and v3.2 (475 rules). Each CVE is characterized by its required SQL structural patterns, which were determined by manually analyzing the minimal proof-of-concept inputs from the CVE disclosures and identifying which grammar non-terminals and production rules are necessary to generate inputs of that structural form.

Table 4.6: CVE reachability analysis. Each CVE requires specific SQL structural patterns that must be expressible within the grammar. Checkmarks indicate the grammar version contains all required building blocks. Grammar v3.2 achieves full reachability for all six target CVEs.

CVE	**Required Patterns**	**v3.1**	**v3.2**
CVE-2019-19646	NOT INDEXED, recursive INSERT	✓	✓
CVE-2020-13434	printf(%lld), large integer literal	✓	✓
CVE-2020-9327	Self-referential generated column (b AS b)	×	✓
CVE-2020-13435	Window function (row_number, lead, lag)	×	✓
CVE-2020-13871	Window function + count() + ORDER BY 3-term	×	✓
CVE-2020-15358	INTERSECT in scalar subquery context	×	✓

Grammar v3.1 achieves full reachability for only two of six target CVEs: CVE-2019-19646 [2], which requires the NOT INDEXED hint and recursive INSERT patterns that were present in the initial grammar design, and CVE-2020-13434 [3], which requires the

printf format string function with a %lld format specifier applied to a large integer literal. Both of these CVEs are triggered by relatively simple SQL constructs that fall within the scope of basic query patterns.

The remaining four CVEs require structural primitives that were absent from v3.1. CVE-2020-9327 [4] requires a self-referential generated column definition where a column's generated expression references the column itself (e.g., `b AS (b) STORED`), creating a circular dependency that triggers a null-pointer dereference during query planning. CVE-2020-13435 [5] and CVE-2020-13871 [6] both require window function syntax (LEAD, LAG, ROW_NUMBER, RANK, and related functions) that exercises SQLite's window function implementation, where use-after-free and buffer overflow vulnerabilities reside. CVE-2020-13871 additionally requires the zero-argument form of the count() aggregate function and an ORDER BY clause with three or more terms to trigger the specific code path containing the vulnerability. CVE-2020-15358 [7] requires an INTERSECT set operation nested within a scalar subquery context, which exercises a rarely-tested interaction between the set operation planner and the scalar expression evaluator.

Grammar v3.2 addresses all four gaps by adding 26 new production rules: 15 window function rules covering the complete set of SQL:2003 window functions (lead, lag, row_number, rank, dense_rank, ntile, first_value, last_value, nth_value, percent_rank, cume_dist, and their OVER clause variations), 3 self-referential generated column expression rules (columns b, c1, and c2 referencing themselves), 3 explicit self-referential column definition list templates with constraints (UNIQUE, NOT NULL), one count() zero-argument form for window-compatible aggregate usage, and one ORDER BY three-term rule for multi-column ordering patterns. With these additions, all six target CVEs have their required structural building blocks present in the grammar, achieving 100% reachability.

This analysis reveals a fundamental insight about grammar-based fuzzing for CVE discovery: vulnerabilities encode specific structural patterns in the input language, not arbitrary byte sequences. A buffer overflow in the window function implementation cannot be triggered without generating syntactically valid SQL that exercises window functions. The grammar therefore acts as a gatekeeping layer — if the grammar cannot express the structural pattern required by a CVE, no amount of fuzzing time or policy sophistication can discover that vulnerability. This makes grammar design the single most critical factor for CVE reachability, superseding both the sampling policy (RQ1) and the campaign duration (RQ2) in importance.

4.5 RQ4: How Does Grammar Evolution Affect Fuzzing Outcomes?

The fourth research question examines how iterative refinement of the grammar — adjusting weights and adding structural primitives — affects the distribution and diversity of discovered crashes.

Table 4.7 summarizes the three grammar versions developed during this work. The evolution proceeded through two distinct phases: weight rebalancing (v3.0 to v3.1) and structural expansion (v3.1 to v3.2).

Table 4.7: Grammar version evolution. Each version addresses a specific deficiency identified through experimental feedback. Rule count is the total number of production rules across all non-terminals.

Version	Rules	CVEs Reachable	Key Change
v3.0	449	2/6	Initial structural primitives (4 statement shapes, 6 schemas, 8 query types)
v3.1	449	2/6	FTS5 schema weight reduced from 2.0 to 0.5
v3.2	475	6/6	+26 rules: window functions, self-ref genCol, count(), ORDER BY 3-term

The transition from v3.0 to v3.1 addressed a weight dominance problem. In v3.0, the Schema-Setup alternative S5 (which generates FTS virtual table schemas) had an initial weight of 2.0, while all other schema alternatives had weight 1.0. This seemingly modest 2:1 ratio produced a dramatic effect: 92% of all crashes discovered during 15-minute campaigns on sqlite-3.31.1 were FTS-related. The FTS module in SQLite is known to contain many shallow bugs that are easily triggered, so the bandit policy's coverage-driven reward mechanism reinforced the already-elevated weight of S5, creating a feedback loop that drove the crash portfolio toward near-total FTS dominance. Reducing the S5 weight to 0.5 in v3.1 immediately diversified the crash portfolio, allowing crashes from generated columns, views, joins, and index operations to appear in campaign results.

The transition from v3.1 to v3.2 addressed structural gaps rather than weight problems.

The CVE reachability analysis (Section 4.4) identified four CVEs that were structurally unreachable due to missing grammar primitives. Adding 26 targeted production rules — specifically chosen to cover the SQL feature space required by the unreachable CVEs — brought reachability from 2/6 to 6/6 without disrupting the existing crash diversity that v3.1's weight rebalancing had achieved.

These results demonstrate that grammar evolution operates on two orthogonal axes. The first axis is weight tuning, which controls the probability distribution over existing rules and thus determines which regions of the input space receive more sampling effort. The second axis is structural expansion, which extends the boundaries of the expressible input space itself. Both are necessary: weight tuning without structural coverage leaves CVEs unreachable regardless of campaign duration, while structural expansion without weight tuning can lead to dominance of one grammar region (as the v3.0 FTS problem illustrates).

The practical implication is that grammar design matters more than sampling policy for CVE discovery. The grammar defines the search space boundaries; the policy merely determines how efficiently that space is navigated. A uniform policy operating on a well-designed grammar (v3.2) outperforms a sophisticated bandit policy operating on a structurally incomplete grammar (v3.1) for the simple reason that structural reachability is a prerequisite for discovery. This finding suggests that future work should prioritize automated grammar expansion (learning new structural primitives from CVE disclosures or code analysis) over increasingly sophisticated sampling policies.

4.6 Threats to Validity

Several threats to validity must be acknowledged when interpreting these results.

Regarding internal validity, the sample size of N=5 runs per (version, policy) cell, while sufficient for non-parametric testing, limits the statistical power to detect small effect sizes. The bandit policy's hyperparameters — specifically the EXP3 learning rate η — were not tuned via grid search or cross-validation but rather set to a single reasonable value based on the literature [17]. A different learning rate might yield better or worse performance relative to uniform sampling, and the optimal rate likely varies across grammar sizes and target programs. Additionally, the coverage logs record crash counts at one-second granularity based on execution order rather than wall-clock timestamps with sub-second precision, which introduces minor measurement noise in the temporal analysis.

Regarding external validity, all experiments target a single DBMS (SQLite [1]) with a single input language (SQL). SQLite is an embedded database with a relatively compact codebase (approximately 150,000 lines of code in the amalgamation), and its bug landscape may not be representative of larger, server-based systems such as MySQL or PostgreSQL. The grammar was manually designed for SQL, and the findings about grammar design importance may not generalize to fuzzing targets that accept less structured input formats. The CVE set is limited to six vulnerabilities across four versions, all discovered between 2019 and 2020. More recent SQLite versions have benefited from extensive fuzzing by the SQLite developers themselves, potentially making the remaining bug population qualitatively different from the CVEs studied here.

Regarding construct validity, the stack-hash deduplication method using the top three frames of the crashing call stack is a heuristic that may both over-count (classifying two manifestations of the same root cause as distinct because they crash through different call paths) and under-count (merging distinct bugs that happen to crash through the same three-frame sequence). The fidelity scoring threshold of 80% — used to filter crashes that reproduce reliably during triage — is an arbitrary cutoff that trades precision for recall. Coverage bitmap hash collisions, where distinct edges map to the same bitmap byte due to the hash function used by AFL-style instrumentation [10], can cause the fuzzer to miss coverage-increasing inputs, potentially affecting both the bandit's reward signal and the crash diversity counts. Finally, the CVE reachability analysis (RQ3) assesses structural expressibility rather than empirical triggering probability; a CVE being reachable in principle does not guarantee it will be triggered within any finite campaign duration.

Conclusion

This thesis presented a grammar-based greybox fuzzing system for automated vulnerability detection in SQLite, integrating structural grammar design, coverage-guided feedback, and adaptive rule selection into a unified pipeline. The system was built on top of the Nautilus 2.0 fuzzer [12] and extended with a multi-armed bandit algorithm for grammar rule weighting. A structural primitives grammar comprising 475 rules across three successive versions was developed through systematic analysis of confirmed CVEs, encoding the SQL constructs most likely to trigger memory safety violations. Experiments spanning 53 fuzzing campaigns across three SQLite versions (3.30.1, 3.31.1, and 3.32.0) — each containing at least one confirmed CVE — were conducted to evaluate whether bandit-driven adaptive rule selection offers an advantage over uniform sampling for CVE-class vulnerability discovery.

Four key findings emerged from the experimental evaluation. First, uniform sampling discovered approximately twice as many unique crash classes as bandit sampling (mean 8.6 versus 4.2, $p \approx 0.01$, Mann-Whitney U test), demonstrating that broad exploration of the grammar space outperforms early exploitation for triggering the diverse bug classes associated with known CVEs. Second, both policies located their first crash within one to two seconds of campaign start, but their discovery rates diverged over time: the bandit's per-minute crash rate decayed by 58% across a campaign, compared to 51% for uniform sampling, indicating that the bandit converged prematurely to a narrow subset of high-reward rules. Third, grammar structural coverage emerged as the primary bottleneck for CVE reachability: four of the six target CVEs were unreachable under earlier grammar versions and became reachable only after grammar v3.2 introduced the necessary SQL primitives, namely window functions and self-referential generated columns. Fourth, and most broadly, grammar evolution through structural expansion had a substantially larger effect on aggregate bug discovery than the choice of sampling policy, because the grammar defines the reachable search space while the policy merely navigates within it [12, 13].

Several limitations bound these findings and should temper the generality of the conclusions. The sample size of five independent runs per experimental cell provides limited statistical power for per-version breakdown, making it difficult to attribute observed differences to specific version characteristics rather than sampling variance. All experiments were conducted on SQLite, and it is not known whether the relative performance of bandit

versus uniform sampling holds for other database management systems with structurally different SQL dialects. The bandit's exponential moving average learning rate was fixed rather than systematically tuned, which may have contributed to its premature convergence and potentially understates the ceiling of bandit performance. Finally, the thesis focused exclusively on grammar rule selection as the adaptive decision and did not implement a full reinforcement learning agent capable of selecting among mutation operators such as random subtree replacement, recursive expansion, and splice.

Three directions warrant further investigation as natural continuations of this work. The most direct extension is the integration of a Deep Q-Network (DQN) agent that learns to select not only grammar rules but also mutation operators based on joint coverage and crash-frequency signals — this constitutes the planned Phase 3 of the RL-Nautilus project and would test whether learned mutation policies can recover the convergence problem observed in the bandit experiments. Second, extending the experimental scope to longer campaign durations of several hours to days, and to additional DBMS targets including MySQL and PostgreSQL, would assess how the relative merits of adaptive versus uniform sampling evolve when the search space is explored more thoroughly and under different grammatical structures. Third, exploring grammar composition techniques that automatically combine structural primitives extracted from multiple CVE analyses could partially automate the grammar design process that this thesis performed manually, potentially closing the loop between vulnerability disclosure and fuzzing grammar construction and making the pipeline more accessible to practitioners without deep SQL internals expertise.

Bibliography

[1] D. Richard Hipp, Dan Kennedy, and Joe Mistachkin. SQLite. `https://www.sqlite.org/`, 2000. Version 3.x.

[2] MITRE. CVE-2019-19646. `https://nvd.nist.gov/vuln/detail/CVE-2019-19646`, 2019. SQLite `expr.c` aggregate query use-after-free.

[3] MITRE. CVE-2020-13434. `https://nvd.nist.gov/vuln/detail/CVE-2020-13434`, 2020. SQLite `printf` integer overflow.

[4] MITRE. CVE-2020-9327. `https://nvd.nist.gov/vuln/detail/CVE-2020-9327`, 2020. SQLite generated column uninitialized pointer.

[5] MITRE. CVE-2020-13435. `https://nvd.nist.gov/vuln/detail/CVE-2020-13435`, 2020.

[6] MITRE. CVE-2020-13871. `https://nvd.nist.gov/vuln/detail/CVE-2020-13871`, 2020.

[7] MITRE. CVE-2020-15358. `https://nvd.nist.gov/vuln/detail/CVE-2020-15358`, 2020.

[8] Valentin J. M. Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. The art, science, and engineering of fuzzing: A survey. *IEEE Transactions on Software Engineering*, 47(11):2312–2331, 2021.

[9] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. AddressSanitizer: A fast address sanity checker. In *Proceedings of the 2012 USENIX Annual Technical Conference (ATC)*, pages 309–318. USENIX Association, 2012.

[10] Michal Zalewski. American fuzzy lop. `https://lcamtuf.coredump.cx/afl/`, 2014.

[11] LLVM Project. libFuzzer – a library for coverage-guided fuzz testing. `https://llvm.org/docs/LibFuzzer.html`.

[12] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Christoph Cloosters, and Ahmad-Reza Sadeghi. NAUTILUS: Fishing for deep bugs with grammars. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2019.

[13] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. Superion: Grammar-aware greybox fuzzing. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 724–735. IEEE / ACM, 2019.

[14] Tim Blazytko, Cornelius Aschermann, Moritz Schlögel, Ali Nawrocki, Steffen Kornmaier, and Thorsten Holz. GRIMOIRE: Synthesizing structure while fuzzing. In *Proceedings of the 28th USENIX Security Symposium*, pages 1985–2002. USENIX Association, 2019.

[15] Rui Zhong, Yongheng Chen, Hong Wen, Hangfan Zhang, Wenke Lee, Dinghao Wu, and Peng Liu. Squirrel: Testing database management systems with language validity and coverage feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 955–970. ACM, 2020.

[16] Andreas Seltenreich. SQLsmith: A random SQL query generator. `https://github.com/anse1/sqlsmith`, 2015.

[17] Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. The nonstochastic multiarmed bandit problem. *SIAM Journal on Computing*, 32(1):48–77, 2002.

[18] Michael D. Vose. A linear algorithm for generating random numbers with a given distribution. *IEEE Transactions on Software Engineering*, 17(9):972–975, 1991.

[19] Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics*, 18(1):50–60, 1947.